



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

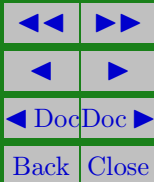
P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

JKUAT SODEL
©2017





ICS 2104 Object-Oriented Programming I

This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 8

Operator Overloading

Learning outcomes

Upon completing this topic you should be able to:

8.1. Operator Overloading

Operator overloading is a mechanism of making user-defined data types to behavior like built-in types. For example, using in-build data types, line 3 below is legal.

```
int x=10; //line 1
```

```
int y=20; //line 2
```

```
int z=x+y; //line 3
```

However, using user-defined data types, line 3 below can not



ICS 2104 Object-Oriented Programming I

be understood by the compiler.

```
Rectangle rect1(4, 6); //line 1
```

```
Rectangle rect2(5, 9); //line 2
```

```
Rectangle rect3=rect1+rect2; //line 3
```

Line 3 will be compiled successfully only if plus(+) operator is overloaded. If the plus(+) operator is overloaded we can be able to add two objects as shown below.

```
Rectangle rect1(10);
```

```
Rectangle rect2(30);
```

```
Rectangle rect3=rect1+rect2;
```

8.2. Operation Overloading Function

In order to overload an operator, we must write a function to do that. Such a function is referred to as an operator overload-



ICS 2104 Object-Oriented Programming I

ing function. The name of an operator overloading function, is reserved word “operator” followed by the operator to overload. For example, to overload $\leq$ (less than or equal to) operator we must write a function with the following name: `operator<=`

The syntax for declaring an operator overloading function is
return-type operator operator-symbol(parameters);

Note the following operators can not be overloaded:

- a) .
- b) .*
- c) ?:
- d) sizeof

An operator overloading function may be either a member function or a non-member function.

Non-member operator overloading functions are made to be



ICS 2104 Object-Oriented Programming I

friend function so that they can access data members of other classes.

8.3. Overloading Binary Operators

Binary operators are those operators that requires at least two operands.

For example:

$+$, $-$, and $*$

Let us now look an example of how to overload $+$ (plus) operator

8.3.1. Overloading $+$ operator using a member function

The name of the operator overloading function in this case is:
`operator+()`



ICS 2104 Object-Oriented Programming I

Recall operator is a reserved word and it is followed by the name of the operator being overloaded, which is + in this case

Now let us assume we have the following statements:

```
Rectangle rect1(6); //line 1
```

```
Rectangle rect2(7); //line 2
```

```
Rectangle rect3=rect1+rect2; // line 3
```

In line 3, we are adding two objects i.e. rect1 and rect2 and the result is assigned to variable rect3 of type Rectangle.

The two objects are of type Rectangle.

If our operator overloading function is a member of Rectangle class,

line 3 will be interpreted by the compiler as shown:

```
Rectangle rect3=rect1.operator+(rect2);
```

Note that on the right side of the assignment operator(=),



ICS 2104 Object-Oriented Programming I

we now have a message operator+ being passed to rect1 object and the object is also being passed argument rect2.

Ideally what we have is a call to a function called operator+() that is being passed rect2 value.

And since the function is a member of Rectangle class, we have to call it using an object that has been created from Rectangle class. Note also that operator+ function must return a value which has to be assigned to rect3 variable.

This value is an object of type Rectangle.

Note that the compiler can not interpret line 3 as shown below `Rectangle rect3=rect2.operator+(rect1);`

Let us now write a complete program that as our operator+() function defined.

/*



ICS 2104 Object-Oriented Programming I

Rectangle.cpp This program demonstrates how binary operators are overloaded

```
#include <iostream.h>
class Rectangle
{
private:
int length;
public:
Rectangle(int );
void print_value(void);
Rectangle operator+(Rectangle);
};
//defining constructor outside
```



ICS 2104 Object-Oriented Programming I

the class declaration

```
Rectangle::Rectangle(int l)
```

```
{ length=l;
```

```
}
```

//defining operator overloading

function outside the class declaration

```
Rectangle Rectangle::operator+(Rectangle r)
```

```
{
```

```
Rectangle rect;
```

```
rect.length=length+r.length; return rect;
```

```
}
```

```
void Rectangle :: print_value(void)
```

```
{
```

```
cout<<"The length is :"<<length <<endl;
```



ICS 2104 Object-Oriented Programming I

```
}  
int main(void)  
{  
    Rectangle rect1(6);  
    Rectangle rect2(7);  
    Rectangle rect3=rect1+rect2;  
    rect1.print_value();  
    rect2.print_value();  
    rect3.print_value();  
    return 0;  
}
```

If you compile and execute this program, it will print the following on the screen





ICS 2104 Object-Oriented Programming I

The length is 6

The length is 7

The length is 13

Note that the syntax of defining an operator overloading function is the same as the syntax of defining other member functions.

8.3.2. Overloading + operator using a non-member function

Let us now see how to overload + operator using a non- member function.

Since the function is a not a member function, it must be a friend function in order for it to access data members.

Consider the three statements below.

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

```
Rectangle rect1(6); //line 1
```

```
Rectangle rect2(7); //line 2
```

```
Rectangle rect3=rect1+rect2; // line 3
```

Since we are overload + operator using a non-member function, the compiler will interpret line 3 as shown below `Rectangle rect3=operator+(rect1, rect2);`

Note that now the two objects are passed as arguments to the operator overloading function.

The operator overloading function returns a value of type `Rectangle`.

Let now write a complete program demonstrating how + operator is overloaded using non member function.

/*

`Rectangle.cpp` This program demonstrates how binary



ICS 2104 Object-Oriented Programming I

operators are overloaded

```
*/ #include <iostream.h>
class Rectangle
{
private:
int length;
public:
Rectangle(int );
void print_value(void);
friend Rectangle operator+(Rectangle);
}; //defining constructor outside
the class declaration
Rectangle::Rectangle(int l)
```



ICS 2104 Object-Oriented Programming I

```
{ length=1;
}
//defining operator overloading function
outside the class declaration
Rectangle operator+(Rectangle r1, Rectangle r2)
{
    Rectangle rect;
    rect.length=r1.length+r2.length;
    return rect;
}
void Rectangle :: print_value(void)
{
    cout<<"The length is :"<<length <<endl;
}
```



ICS 2104 Object-Oriented Programming I

```
int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2(7);
    Rectangle rect3=rect1+rect2;
    rect1.print_value();
    rect2.print_value();
    rect3.print_value();
    return 0;
}
```

If you compile and execute this program, it will produce the following output on the screen

The length is 6



The length is 7

The length is 13

8.4. Overloading Unary Operators

Unary operators are associated with only one operand.

Increment and decrement operators are examples of unary operators.

These operators are overloaded just like binary operators.

The overloading function can be either a member function or a non-member function.

Increment and decrement operators can be either pre-or post decrement and increment operators



8.4.1. Overloading pre-increment/decrement operators using member functions

Consider the statement shown below:

```
Square square1(20); //line 1
```

```
Square square2=++square1; //line 2
```

Since we are overloading ++ operator using a member function line 2 will be interpreted by the compiler as follows: `Square square2=square1.operator++()`;

What is on the right side of assignment operator (=) is a call to `operator++()` function.

In an object-oriented program it is message passing. `square1` is the object sent the message, and `operator++` is the message being passed.

Below is an example of a program that shows how pre-



ICS 2104 Object-Oriented Programming I

increment operation is overloaded using a member function

/*

Rectangle.cpp This program demonstrates how binary operators are overloaded

```
*/ #include <iostream.h>
```

```
class Rectangle
```

```
{
```

```
private:
```

```
int length;
```

```
public:
```

```
Rectangle(int);
```

```
void print_value(void);
```

```
Rectangle operator++();
```



ICS 2104 Object-Oriented Programming I

```
}; //defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{ length=l;
}

//defining operator overloading function
outside the class declaration Rectangle Rectangle
::operator++()
{
length++;
return *this;
}

void Rectangle :: print_value(void)
{
cout<<"The length is :"<<length <<endl;
```



ICS 2104 Object-Oriented Programming I

```
}  
int main(void)  
{  
    Rectangle rect1(6);  
    Rectangle rect2=++rect1;  
    rect1.print_value();  
    rect2.print_value();  
    return 0;  
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7

The length is 7



8.5. Overloading pre-increment/decrement operators using a non-member function

Consider the statement shown below:

```
Square square1(20); //line 1
```

```
Square square2=++square1; //line 2
```

Since we are overloading ++ operator using a non-member function line 2 will be interpreted by the compiler as follows:

```
Square square2=operator++(square1);
```

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is referred to as message passing.

square1 is the object that is passed to the operator++() method.



ICS 2104 Object-Oriented Programming I

Below is an example of a program that shows how pre-increment operation is overloaded using a non member function

/*
Rectangle.cpp This program demonstrates
how binary operators are overloaded

```
*/ #include <iostream.h>
class Rectangle
{
    private:
        int length;
    public:
        Rectangle(int);
        void print_value(void);
```



ICS 2104 Object-Oriented Programming I

```
friend Rectangle operator++();  
  
};  
//defining constructor outside the class declaration  
Rectangle::Rectangle(int l)  
{  
    length=l;  
}  
//defining operator overloading function  
outside the class declaration  
Rectangle operator++(Rectangle rect)  
{  
    (rect.length)++;  
    return *rect;  
}
```



ICS 2104 Object-Oriented Programming I

```
void Rectangle :: print_value(void)
{
    cout<<"The length is :"<<length <<endl;
}

int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2=++rect1;
    rect1.print_value();
    rect2.print_value();
    return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

JKUAT: Setting trends in higher Education, Research and Innovation



The length is 7

The length is 7

8.6. Overloading post-increment/decrement operators using member functions

Consider the statement shown below:

```
Square square1(20); //line 1
```

```
Square square2=square1++; //line 2
```

Since we are overloading ++ operator using a member function line 2 will be interpreted by the compiler as follows: Square square2=square1.operator++(0);

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is message passing. Zero



ICS 2104 Object-Oriented Programming I

(0) is a value that is sent to operator++() function.

Zero (0) is an argument that is used to distinguish between pre and post decrement/increment operators.

Zero (0) has no other use.

In fact it is referred to as a dummy parameter.

Below is an example of a program that shows how post-increment operation is overloaded using a member function

```
/*  
Rectangle.cpp This program demonstrates  
how binary  
operators are overloaded
```

```
*/ #include <iostream.h>  
class Rectangle
```



ICS 2104 Object-Oriented Programming I

```
{
    private:
        int length;
    public:
        Rectangle(int );
        void print_value(void);
        Rectangle operator++(int );
};
//defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{
    length=l;
}
//defining operator overloading function
```



ICS 2104 Object-Oriented Programming I

outside the class declaration

```
Rectangle Rectangle :: operator++(int dummy)
{
    Rectangle temp=*this;
    length++;
    return temp;
}

void Rectangle :: print_value(void)
{
    cout<<"The length is "<<length <<endl;
}

int main(void)
{
    Rectangle rect1(6);
```



ICS 2104 Object-Oriented Programming I

```
Rectangle rect2=rect1++;  
rect1.print_value();  
rect2.print_value();  
return 0;  
}
```

If you compile and execute this program, it will output the following on the screen.

The length is7

The length is 8

8.7. Overloading post-increment/decrement operators using non-member function

Consider the statement shown below:

Square square1(20); //line 1



ICS 2104 Object-Oriented Programming I

```
Square square2=square1++; //line 2
```

Since we are overloading ++ operator using a non-member function line 2 will be interpreted by the compiler as follows:
Square square2=operator++(square1, 0);

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is message passing.

square1 and zero (0) are values that are sent to operator++() function. Zero (0) is used to distinguish between pre and post decrement/increment operator overloading.

Zero (0) has no other use.

Below is an example of a program that shows how post-increment operation is overloaded using a non-member function

/*



ICS 2104 Object-Oriented Programming I

Rectangle.cpp This program demonstrates how unary operator is overloaded

```
*/ #include <iostream.h>
class Rectangle
{
    private:
        int length;
    public:
        Rectangle(int );
        void print_value(void);
        friend Rectangle operator++(Rectangle);
};
```



ICS 2104 Object-Oriented Programming I

```
//defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{
    length=l;
}
//defining operator overloading function outside
the class declaration Rectangle operator
++(Rectangle rect, int dummy)
{
    Rectangle temp=*this;
    (rect.length)++;
    return temp;
}
void Rectangle :: print_value(void)
```



ICS 2104 Object-Oriented Programming I

```
{
    cout<<"The length is :"<<length <<endl;
}

int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2=rect1++;
    rect1.print_value();
    rect2.print_value();
    return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7



The length is 8

8.8. Overloading stream insertion (<<) and extraction (>>) operators

Stream insertion(<<) and extraction (>>) operators are overloaded using non-member functions.

8.8.1. Overloading the stream insertion (<<) operators

Suppose our insertion(<<) operator is being overloaded for Rectangle class.

Consider the following statements:

```
Rectangle rect; //line 1
```

```
cin>>rect; //line 2
```

```
cout<<rect; //line 3
```



ICS 2104 Object-Oriented Programming I

Line 2 will be interpreted as follows by the compiler as:
`operation>>(cin, rect);`

This statement is a call to `operator>>()` function and the function is passed two arguments: `cin` and `rect`.

Line 3 will be interpreted as follows by the compiler as:
`operator<<(cout, rect);`

This is a call to `operator<<()` function and the function is passed two arguments: `cout` and `rect`.

Note

1. `cin` and `cout` are objects of type `istream` and `ostream` respective.

2. `operators<<()` and `operator>>()` functions return objects of type `ostream` and `istream` respectively.

Below is an example of a program, that demonstrates how



ICS 2104 Object-Oriented Programming I

insertion (<<) and extraction (>>) operators are overloaded

```
/*  
overload.cpp This program demonstrates  
how to overload insertion (<<)  
and extraction(>>) operators
```

```
*/  
\#include<iostream.h>  
class Rectangle  
{  
    private:  
        int length;  
    public:  
        Rectangle(int);
```



ICS 2104 Object-Oriented Programming I

```
friend ostream& operator<<
(ostream& output, Rectangle rect1);
friend istream& operator>>
(istream& input, Rectangle rect2);
};
//defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{
    length=l;
}
//defining operator overloading
function outside the class declaration
ostream& operators<<(ostream& output,
Rectangle rect1)
```



ICS 2104 Object-Oriented Programming I

```
{
    output<<rect1.length <<endl; return output;
}
istream& operators<<(istream& input, Rectangle rec)
{
    rect2>>rect2.length;
    return input;
}
int main(void)
{
    Rectangle rect1(6);
    cout<<"The value is "<<rect1<<endl;
    cout<<"Enter a value: ";
    cin>>rect1 cout<<"The value is "<<rect1<<endl;
```



ICS 2104 Object-Oriented Programming I

```
    return 0;  
}
```

If you compile and execute this program, the following will be displayed on the computer screen.

The value is 6 Enter a value: 10 The value is 10

Revision Questions

EXERCISE 1. ✎ Define “friend function”

Example ✎.

Suppose that the operator `<<` is to be overloaded for a user-defined class called `mystery`. Why must `<<` operator be overloaded using a friend operator overloading function

Solution: The leftmost operand of `<<` is an object of type `os-`



ICS 2104 Object-Oriented Programming I

stream, and therefore since the class is of type `mystery` an object of type `ostream` can not be used to call the operator overloading function. This object has to be passed to the function as an argument. In that case the operator overloading function must be a friend operator overloading function □

EXERCISE 2. ✎ List any FOUR characteristics of a friend function

EXERCISE 3. ✎ Explain the use of “this” keyword

EXERCISE 4. ✎ Consider the following:

```
Rectangle rect(4,6); //line 1
```

```
cout<<rect; //line 2
```

show how the compiler will interpret line 2



References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)



Solutions to Exercises

Exercise 1. A friend function is a function that is not a member of a class but it has access to the data member of the class

Exercise 1